Audio Content-Based Playlist Generation

Frederic Vaquer

April 26, 2026

Work can also be found in the following github repo (json file with the computed embeddings not provided there): `https://github.com/fredericVaquer/audio-content-based-playlists`

1 Audio Analysis

Audio is loaded once per file using Essentia's `AudioLoader`, which yields a stereo signal at the native sample rate. The stereo signal is passed directly to `LoudnessEBUR128`, then downmixed with `MonoMixer` and fed into two `Resample` instances — one targeting 16 kHz for the Discogs-Effnet TensorFlow models, one targeting 48 kHz for CLAP. The resamplers are instantiated fresh for each file with the correct `inputSampleRate` to ensure correct resampling across files with different native sample rates, rather than reusing a preconfigured instance, which is something I tried by defining the resampler before the main loop with the output sample rate and only reconfiguring the input sample rate at every file, but the audio effect was not the expected one as the classifiers on the EffnetDiscogs did not work properly (Electronic music dominance and wrong voice vs instrumental distribution).

All requested descriptors were extracted: tempo via `TempoCNN`, key and scale with `KeyExtractor` under the `temperley`, `krumhansl`, and `edma` profiles, integrated loudness in LUFS, Discogs-Effnet embeddings reused for the Genre Discogs400, voice/instrumental, and danceability classifiers, and CLAP embeddings via the Hugging Face `transformers` library as in the github it is mentioned that it is supported there. For danceability, the Discogs-Effnet-based classifier was used over the signal processing alternative. All classifier outputs are averaged across frames to produce a single prediction per track. Moreover, all the models have been loaded once before starting the loop over the music collection, so that the model is not loaded for every single file.

Results are stored in a single JSON file mapping each track path to its descriptors. To avoid the overhead of writing to disk on every track, results are flushed every 10 files. Each file is processed inside a `try/except` block so that errors are printed and skipped without interrupting the run. On restart, the script checks whether a file path is already present in the JSON before processing it, avoiding redundant computation.

2 Music Collection Overview

For each track, the music style is determined as the Discogs400 class with the highest activation value. Parent broad genres and full per-style counts are reported in the accompanying `style_distribution.tsv`.

The collection is reasonably diverse across broad genres. Rock is the most represented parent genre (540 tracks), followed closely by Electronic (470), Hip Hop (300), Folk/World & Country (200), and Pop (175). Smaller but present categories include Latin, Classical, Funk/Soul, Jazz,

Reggae, and Blues. Rock and Electronic together account for nearly half of all tracks, but the remaining half is spread across a wide variety of styles, suggesting genuine cross-genre coverage.

Tempo is spread between 60 and 185 BPM with a roughly bimodal shape: a first cluster around 90 BPM and a more prominent peak around 125–130 BPM, characteristic of electronic dance music. The danceability distribution shows a strong U-shape, with a large spike near 0 and an even larger concentration near 1. This likely reflects the classifier's tendency toward confident binary predictions, and suggests the danceability score may be better used as a binary filter than a continuous range.

The tonality plots show that the three profiles — Temperley, Krumhansl, and Edma — produce broadly similar key distributions but with visible differences. Temperley and Krumhansl agree closely on major keys, particularly C major and F major, while Edma distributes counts more evenly and assigns noticeably more tracks to minor keys, especially A minor. The percentage of tracks on which all three profiles agree is [**49.10%**]. We select the **Temperley** profile for the UI, as it is the most established profile for Western tonal music and produces the most concentrated, interpretable distribution.

Integrated loudness ranges from approximately −20 to −6 LUFS, peaking around −9 to −10 LUFS. This is louder than the −14 LUFS streaming normalisation target marked in the figure, which is expected: tracks from music catalogues are often mastered loud for commercial reasons, and the MusAV previews are no exception.

The vocal/instrumental split at the default threshold of 0.5 gives 71.4% vocal and 28.6% instrumental. The threshold analysis shows a gradual curve rather than a sharp transition, indicating that a substantial portion of tracks sit in an ambiguous range where the classifier is not highly confident. This is plausible for a genre-diverse collection where many tracks contain processed or sampled material that is difficult to classify cleanly.

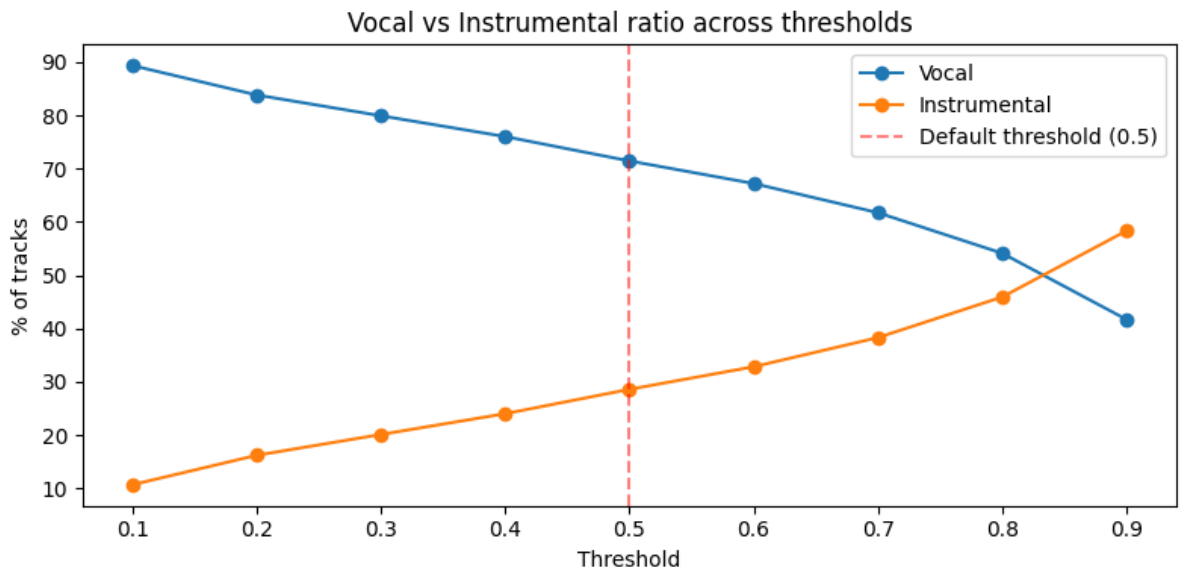

Figure 1: Statistical overview of the MusAV collection.

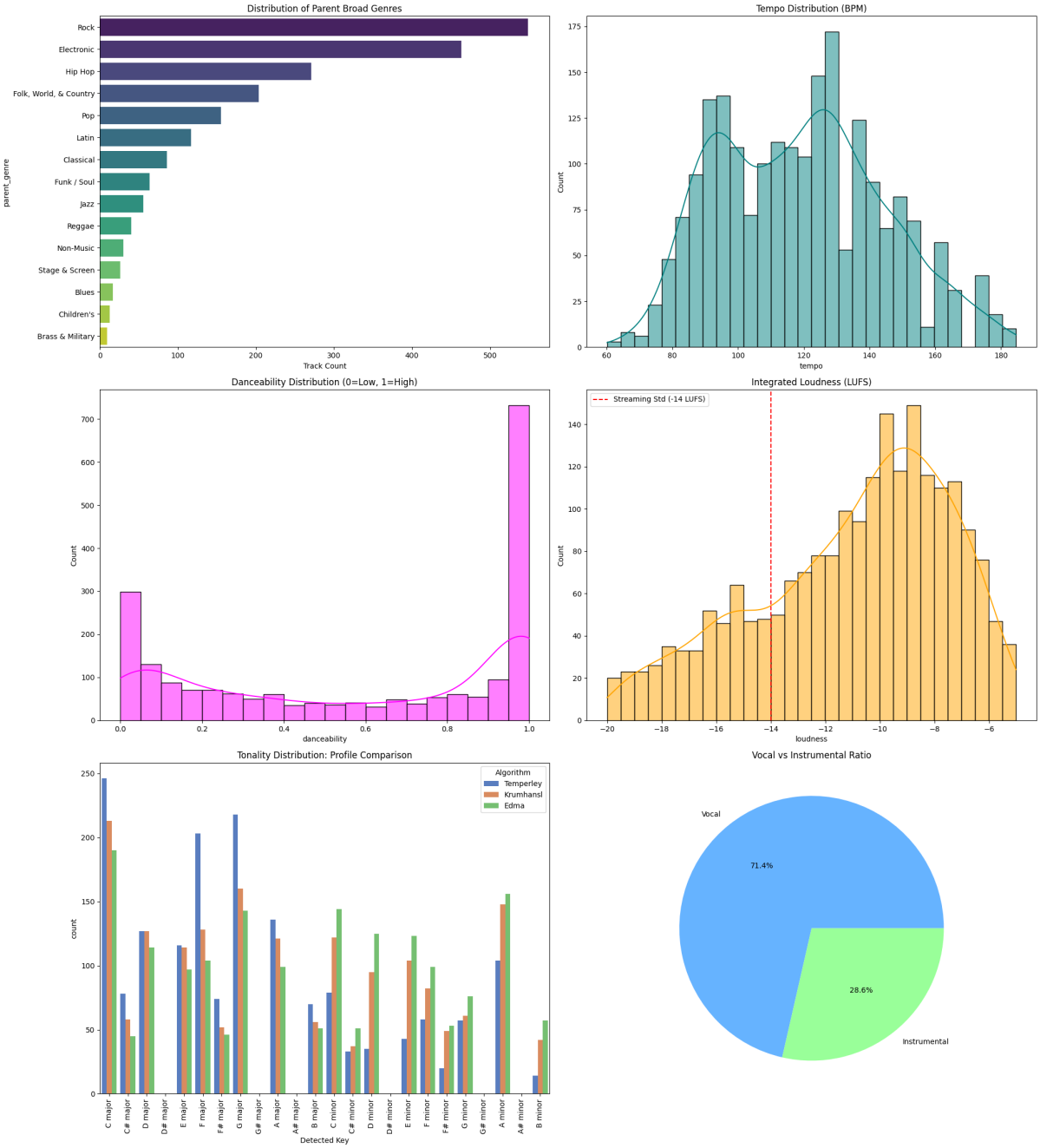

Figure 2: Statistical overview of the MusAV collection.

3 Playlist Generation

Instead of building 3 different apps, I decided to actually build a single app, with 3 different research modes. I felt it more convenient in terms of UI for the user, of course. However, it's true that one of the downsides of doing it this way is that if someone does not want to do free text query search, the app loads the CLAP model for no reason. Still, I thought that having everything on a single app instead of 3 apps made navigation for the user across apps easier.

3.1 Descriptor-Based Filtering

The descriptor filtering UI provides sliders for tempo range and danceability, a radio button for voice/instrumental, and dropdowns for key and scale using the Temperley profile. Results are displayed as the top 10 matching tracks with embedded audio players, and the full filtered playlist can be exported as an M3U8 file.

In terms of quality, the most notable issue is with tempo estimation: octave errors are common, where a track is detected at half or double its perceptual tempo (double is more common here, in higher bpm). This is a known limitation of tempo estimators on diverse-genre collections, where the periodicity of the beat can be ambiguous between a half-time and double-time interpretation.

Danceability filtering behaves unexpectedly near the low end of the range (values close to 0). Tracks filtered at very low danceability do not always sound undanceable, suggesting the classifier is either overconfident or that the low-activation edge of the distribution contains misclassified tracks. The filter works more reliably in the mid-to-high range. I must state though that most of the non danceable filtered samples I have come across are slow danceable rather than "frenetic" dance to put it in a way.

Key and scale estimation performs reasonably well overall, particularly scale (major/minor) detection, which is mostly correct. The main edge case is non-music content such as sound effects or ambient tracks, which receive a detected key despite having no clear tonal center. This is an inherent limitation of the algorithm rather than a bug.

3.2 Similarity-Based Search

The similarity interface allows selecting a query track from a dropdown, then displays the 10 most similar tracks computed separately using Effnet-Discogs and CLAP mean embeddings, both via cosine similarity. Results are shown side by side with embedded audio players for direct comparison.

Overall, both models perform surprisingly well. A notable observation is that the two lists frequently share tracks, which is expected for stylistically prototypical examples where both embeddings agree on similarity. Divergence between the two appears at the edges, where a track can be similar either in style or in acoustic texture but not necessarily both. Effnet-Discogs, being trained on Discogs genre labels, organizes its embedding space primarily around genre and subgenre boundaries, and retrieval results reflect this: similar tracks tend to share tempo, mode, and overall stylistic character. CLAP, trained on heterogeneous audio-text pairs, appears more sensitive to timbral and acoustic surface features. In vocal tracks, CLAP occasionally surfaces tracks in the same language, likely because vocal phonetics and timbre create a consistent acoustic fingerprint that the model encodes. More generally, CLAP seems to capture a broader notion of "sounds like" rather than "belongs to the same genre as", which makes it complementary to Effnet rather than strictly better or worse.

3.3 Text Query Search

The text search interface takes a free-form text input, encodes it with the CLAP text encoder, and retrieves the 10 most similar tracks by cosine similarity against the stored CLAP audio embeddings.

The results are generally impressive. Specific and acoustically grounded queries work best: *"upbeat electronic dance music with synthesizers"* returns consistently relevant results across all 10 tracks, *"hip hop with heavy bass and rap vocals"* is near perfect, and *"slow sad piano melody"* retrieves well-matched results. Queries combining mood and instrument such as *"acoustic guitar fingerpicking"* work mostly well, though ranking is imperfect — tracks where fingerpicked guitar

is the most prominent element appear mid-list rather than at the top. *"Female vocals with reverb"* retrieves tracks with the correct reverberant quality, though some results include choir arrangements with male voices, suggesting the model encodes the reverb texture more reliably than the gender of the voice. One result is purely instrumental with a faint humming-like sound, indicating the model occasionally latches onto loose acoustic resemblances.

Functional and contextual queries also perform reasonably. *"Music for studying"* returns calm, low-energy tracks including Pachelbel's Canon, which could be argued as among the best results, though tracks ranked 1–3 are arguably too energetic for the use case — a reflection of how subjective such queries are rather than a clear model failure. Interestingly, none of the results contain lyrics, which aligns with common expectations for study music. *"Music from the 1970s"* works better than expected given that era is not a direct acoustic feature, likely because CLAP has learned associations between decade and production style from its training captions. Single-word genre queries such as *"jazz"* and *"classical"* also work well despite these genres being underrepresented in the collection.

The clearest failure is *"Happy Birthday song"*, which returns unrelated results, likely because the query is too culturally specific and the song itself is absent from the collection. *"Music in Spanish"* retrieves Spanish-language tracks for the top results but breaks down toward ranks 9–10, which is not very suprising as maybe only 8 Spanish-language tracks may exist in the entire collection — the model simply runs out of relevant material.

4 Conclusion

This project demonstrates a functional end-to-end content-based music retrieval system applied to the MusAV collection. Audio descriptors extracted with Essentia and CLAP cover a broad range of musical attributes — tempo, key, loudness, danceability, voice presence, genre, and semantic embeddings — and are exposed through three complementary retrieval interfaces. Descriptor-based filtering works well for most attributes, with tempo octave errors and danceability edge cases as the main limitations. Both embedding-based similarity approaches perform surprisingly well, with Effnet-Discogs capturing genre-level coherence and CLAP capturing timbral and acoustic surface similarity, making them complementary rather than redundant. Text-based search is the most impressive of the three modes, handling acoustically grounded queries reliably and even managing contextual queries such as era or use case with reasonable results, with failures mostly limited to culturally specific or out-of-vocabulary queries.

Several directions could improve the system. On the feature side, adding mood or emotion descriptors and a more robust tempo estimator with octave-error correction would address two of the most noticeable failure modes. On the UI side, combining multiple retrieval modes — for example, using a text query as a starting point and then refining by descriptor sliders — would make the interface significantly more powerful. Allowing the user to select multiple query tracks for similarity search and aggregating their embeddings could also improve discovery. Finally, displaying richer metadata alongside each result (detected genre, key, tempo, danceability score) directly in the audio player cards would help the user understand why a track was retrieved and build more informed playlists.